In [117…

```
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn_extra.cluster import KMedoids
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
```

In [118…

```
data_set.head()
```

Out[118]:

	CUST_ID	BALANCE	BALANCE_FREQUENCY	PURCHASES	ONEOFF_PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY	ONEOFF_PURCHASES_FREQUENCY	PURCHASES_INSTALLMEN
0	C10001	40.900749	0.818182	95.40	0.00	95.4	0.000000	0.166667	0.000000	
1	C10002	3202.467416	0.909091	0.00	0.00	0.0	6442.945483	0.000000	0.000000	
2	C10003	2495.148862	1.000000	773.17	773.17	0.0	0.000000	1.000000	1.000000	
3	C10004	1666.670542	0.636364	1499.00	1499.00	0.0	205.788017	0.083333	0.083333	
4	C10005	817.714335	1.000000	16.00	16.00	0.0	0.000000	0.083333	0.083333	

Il dataset analizzato contiene informazioni anonime sui titolari di carte di credito, con lo scopo di comprendere i comportamenti di spesa e di utilizzo della carta. Di seguito si riportano le variabili presenti nel dataset, con una breve spiegazione del loro significato:

CUST_ID: identificativo univoco del titolare della carta di credito. È una variabile categoriale e non viene utilizzata per l'analisi, poiché serve unicamente per identificare l'osservazione.

BALANCE: saldo residuo disponibile sul conto, ovvero l'importo che l'utente ha ancora a disposizione per effettuare acquisti.

BALANCE_FREQUENCY: frequenza con cui il saldo viene aggiornato. È un punteggio compreso tra 0 e 1, dove 1 indica un aggiornamento molto frequente, mentre 0 rappresenta un aggiornamento poco frequente.

PURCHASES: importo totale degli acquisti effettuati con la carta.

ONEOFF_PURCHASES: importo massimo speso in un singolo acquisto (ossia acquisti "una tantum").

INSTALLMENTS_PURCHASES: importo totale degli acquisti effettuati a rate.

CASH_ADVANCE: ammontare di denaro prelevato in anticipo (anticipo di contante) da parte dell'utente.

PURCHASES_FREQUENCY: frequenza con cui vengono effettuati acquisti. Anche questa variabile assume valori tra 0 e 1, dove 1 indica acquisti frequenti.

ONEOFFPURCHASESFREQUENCY: frequenza con cui vengono effettuati acquisti "una tantum". Il valore è compreso tra 0 (mai) e 1 (molto frequentemente).

PURCHASESINSTALLMENTSFREQUENCY: frequenza degli acquisti effettuati a rate. Anch'essa varia tra 0 e 1.

CASHADVANCEFREQUENCY: frequenza con cui vengono richiesti anticipi di contante (valori tra 0 e 1).

CASHADVANCETRX: numero di transazioni effettuate tramite anticipo di contante.

PURCHASES_TRX: numero totale di transazioni di acquisto effettuate dall'utente.

CREDIT_LIMIT: limite massimo di credito disponibile sulla carta.

PAYMENTS: importo totale dei pagamenti effettuati dall'utente.

MINIMUM_PAYMENTS: importo minimo dei pagamenti effettuati, ovvero il minimo richiesto dalla banca per non incorrere in sanzioni.

PRCFULLPAYMENT: percentuale del debito che l'utente ha pagato completamente (valori tra 0 e 1).

TENURE: durata, espressa in mesi, del rapporto tra l'utente e la carta di credito (es. da quanti mesi possiede la carta).

In [119…

```
# CONTROLLIAMO SE LE FEATURES SONO CORRETTAMENTE CODIFCATE
data_set.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8950 entries, 0 to 8949
Data columns (total 18 columns):
 #   Column                            Non-Null Count  Dtype  
---  ------                            --------------  -----  
 0   CUST_ID                           8950 non-null   object 
 1   BALANCE                           8950 non-null   float64
 2   BALANCE_FREQUENCY                 8950 non-null   float64
 3   PURCHASES                         8950 non-null   float64
 4   ONEOFF_PURCHASES                  8950 non-null   float64
 5   INSTALLMENTS_PURCHASES            8950 non-null   float64
 6   CASH_ADVANCE                      8950 non-null   float64
 7   PURCHASES_FREQUENCY               8950 non-null   float64
 8   ONEOFF_PURCHASES_FREQUENCY        8950 non-null   float64
 9   PURCHASES_INSTALLMENTS_FREQUENCY  8950 non-null   float64
 10  CASH_ADVANCE_FREQUENCY            8950 non-null   float64
 11  CASH_ADVANCE_TRX                  8950 non-null   int64  
 12  PURCHASES_TRX                     8950 non-null   int64  
 13  CREDIT_LIMIT                      8949 non-null   float64
 14  PAYMENTS                          8950 non-null   float64
 15  MINIMUM_PAYMENTS                  8637 non-null   float64
 16  PRC_FULL_PAYMENT                  8950 non-null   float64
 17  TENURE                            8950 non-null   int64  
dtypes: float64(14), int64(3), object(1)
memory usage: 1.2+ MB
```

SI, LE FEATURES SONO DI TIPO FLOAT64 E INT64

In [120…

```
# EVENTUALI VALORI MANCANTI (NA)

print("Valori mancanti per colonna:")
print(data_set.isna().sum())
```

```
Valori mancanti per colonna:
CUST_ID                               0
BALANCE                               0
BALANCE_FREQUENCY                     0
PURCHASES                             0
ONEOFF_PURCHASES                      0
INSTALLMENTS_PURCHASES                0
CASH_ADVANCE                          0
PURCHASES_FREQUENCY                   0
ONEOFF_PURCHASES_FREQUENCY            0
PURCHASES_INSTALLMENTS_FREQUENCY      0
CASH_ADVANCE_FREQUENCY                0
CASH_ADVANCE_TRX                      0
PURCHASES_TRX                         0
CREDIT_LIMIT                          1
PAYMENTS                              0
MINIMUM_PAYMENTS                    313
PRC_FULL_PAYMENT                      0
TENURE                                0
dtype: int64
```

In [121…

```
# ELIMINIAMO I VALORI MANCANTI

data_set_clean = data_set.dropna().copy()
```

In [122…

```
print(data_set_clean.isna().sum())
```

```
CUST_ID                             0
BALANCE                             0
BALANCE_FREQUENCY                   0
PURCHASES                           0
ONEOFF_PURCHASES                    0
INSTALLMENTS_PURCHASES              0
CASH_ADVANCE                        0
PURCHASES_FREQUENCY                 0
ONEOFF_PURCHASES_FREQUENCY          0
PURCHASES_INSTALLMENTS_FREQUENCY    0
CASH_ADVANCE_FREQUENCY              0
CASH_ADVANCE_TRX                    0
PURCHASES_TRX                       0
CREDIT_LIMIT                        0
PAYMENTS                            0
MINIMUM_PAYMENTS                    0
PRC_FULL_PAYMENT                    0
TENURE                              0
dtype: int64
```

In [123…

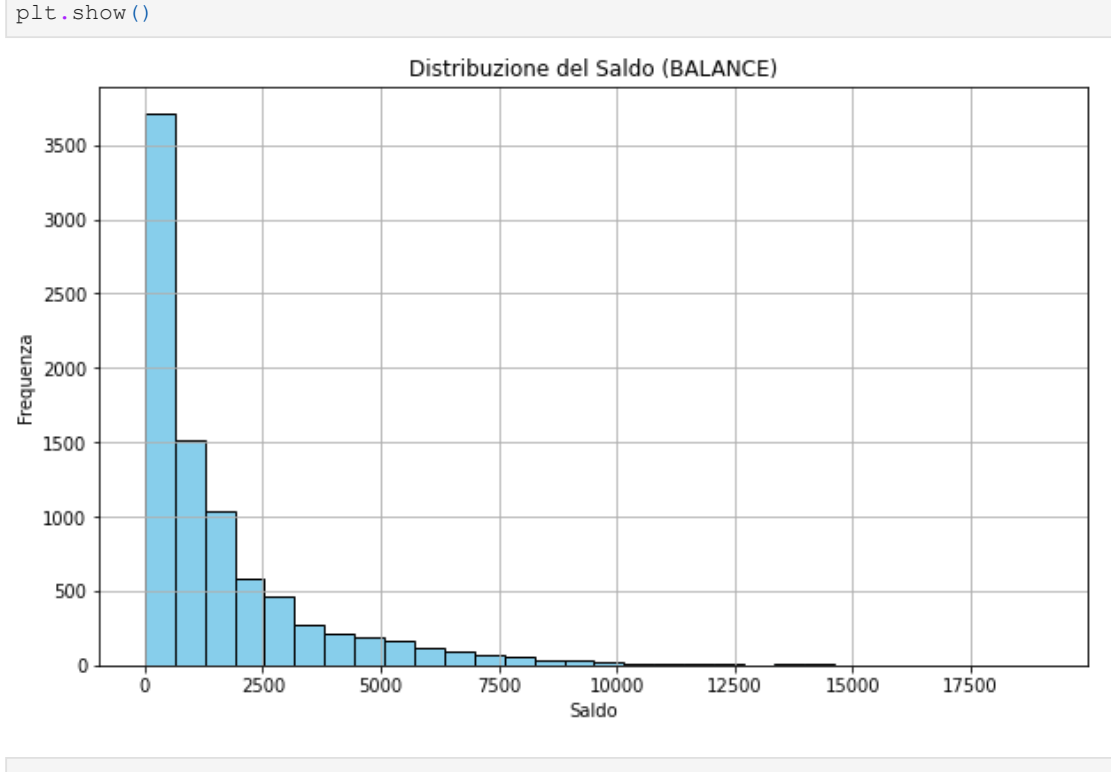
```
data_set_clean.duplicated().sum()# Non ci sono righe duplicate
```

Out[123]:

```
0
```

In [55]:

```
# BALANCE: saldo residuo disponibile sul conto, ovvero l'importo che l'utente ha ancora a disposizione per effettuare acquisti.

# Istogramma della variabile BALANCE
plt.figure(figsize=(10, 6))  # Dimensione grafico
plt.hist(data_set_clean['BALANCE'], bins=30, color='skyblue', edgecolor='black')
plt.title("Distribuzione del Saldo (BALANCE)")
plt.xlabel("Saldo")
plt.ylabel("Frequenza")
plt.grid(True)
plt.show()
```



In [124…

```
data_set_clean[['BALANCE']].describe()
```

Out[124]:

	BALANCE
count	8636.000000
mean	1601.224893
std	2095.571300
min	0.000000
25%	148.095189
50%	916.855459
75%	2105.195853
max	19043.138560

BALANCE_FREQUENCY: frequenza con cui il saldo viene aggiornato. È un punteggio compreso tra 0 e 1, dove 1 indica un aggiornamento molto frequente, mentre 0 rappresenta un aggiornamento poco frequente.

In [125…

```
data_set_clean['BALANCE_FREQUENCY'] = pd.cut(
    data_set_clean['BALANCE_FREQUENCY'],
    bins=[0, 0.5, 1],
    labels=['Basso', 'Alto'],
    include_lowest=True
).astype('object')
```

In [126…

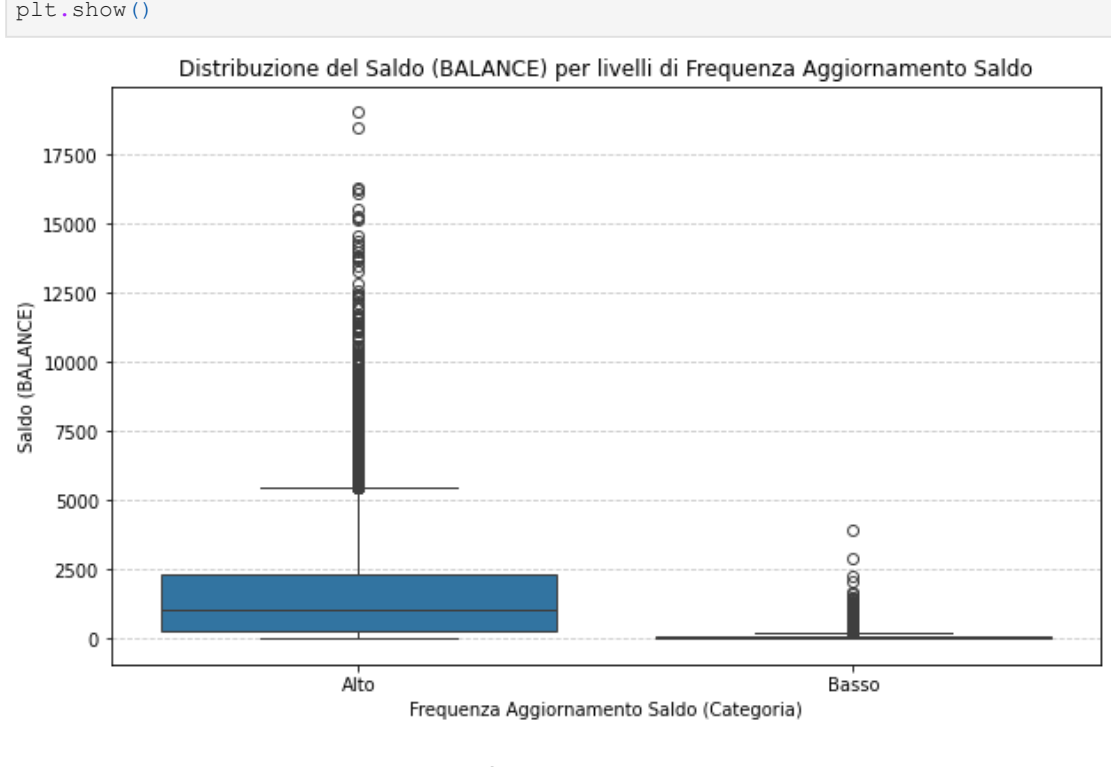
```
data_set_clean['BALANCE_FREQUENCY'].head()
```

Out[126]:

```
0    Alto
1    Alto
2    Alto
4    Alto
5    Alto
Name: BALANCE_FREQUENCY, dtype: object
```

In [127…

```
plt.figure(figsize=(10,6))
sns.boxplot(x='BALANCE_FREQUENCY', y='BALANCE', data=data_set_clean, hue='BALANCE_FREQUENCY')
plt.title('Distribuzione del Saldo (BALANCE) per livelli di Frequenza Aggiornamento Saldo')
plt.xlabel('Frequenza Aggiornamento Saldo (Categoria)')
plt.ylabel('Saldo (BALANCE)')
plt.grid(True, axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Notiamo che i clienti che aggiornano più di frequente il saldo, sono anche coloro che presentano un saldo molto maggiore chi ha un saldo minore tende a risparmiare maggiormente, dunque la frequenza di aggiornamento del saldo sarà minore.

Dunque possiamo pensare che chi aggiorna più di frequente il saldo e che presenta un saldo maggiore, farà anche piu acquisti con una maggiore frequenza, quindi avrà un numero di transazioni maggiore. Scopriamolo.

In [60]:

```
data_set_clean.groupby('BALANCE_FREQUENCY')[['PURCHASES', 'PURCHASES_FREQUENCY', 'PURCHASES_TRX']].mean()
```

Out[60]:

	PURCHASES	PURCHASES_FREQUENCY	PURCHASES_TRX
BALANCE_FREQUENCY			
Alto	1089.049494	0.519828	16.063482
Basso	350.629476	0.243240	4.104839

I clienti il cui saldo viene aggiornato con bassa frequenza presentano, in media, un importo totale degli acquisti effettuati con la carta di credito significativamente inferiore rispetto a quelli che aggiornano il saldo con maggiore frequenza.

Inoltre, considerando la frequenza con cui vengono effettuati gli acquisti, si osserva che chi acquista più frequentemente tende anche a sostenere spese più elevate e ad aggiornare il saldo più spesso.

PURCHASES_TRX nella tabella rappresenta la media del numero totale di transazioni di acquisto effettuate dal cliente. Il tutto è coerente con quanto detto in precedenza.

In [61]:

```
# INSTALLMENTS_PURCHASES: importo totale degli acquisti effettuati a rate.

# PURCHASESINSTALLMENTSFREQUENCY: frequenza degli acquisti effettuati a rate. Varia tra 0 e 1.

data_set_clean.groupby('BALANCE_FREQUENCY')[['INSTALLMENTS_PURCHASES', 'PURCHASES_INSTALLMENTS_FREQUENCY']].mean()
```

Out[61]:

	INSTALLMENTS_PURCHASES	PURCHASES_INSTALLMENTS_FREQUENCY
BALANCE_FREQUENCY		
Alto	447.988685	0.387354
Basso	132.900605	0.172223

In [62]:

```
# CREDIT_LIMIT: limite massimo di credito disponibile sulla carta.

data_set_clean.groupby('BALANCE_FREQUENCY')[['CREDIT_LIMIT']].mean()
```

Out[62]:

	CREDIT_LIMIT
BALANCE_FREQUENCY	
Alto	4589.668485
Basso	3805.261350

I clienti che aggiornano spesso il saldo, presentano in media un limite massimo di credito disponibile sulla carta maggiore.

In [63]:

```
# PAYMENTS: importo totale dei pagamenti effettuati dall'utente.

# MINIMUM_PAYMENTS: importo minimo dei pagamenti effettuati, ovvero il minimo richiesto dalla banca per non incorrere in sanzioni.

# PRCFULLPAYMENT: percentuale del debito che l'utente ha pagato completamente (valori tra 0 e 1).


data_set_clean.groupby('BALANCE_FREQUENCY')[['PAYMENTS', 'MINIMUM_PAYMENTS', 'PRC_FULL_PAYMENT']].mean()
```

Out[63]:

	PAYMENTS	MINIMUM_PAYMENTS	PRC_FULL_PAYMENT
BALANCE_FREQUENCY			
Alto	1808.296454	931.042115	0.150937
Basso	1531.824264	156.388598	0.248055

I clienti che aggiornano frequentemente il saldo (BALANCE_FREQUENCY = alto) presentano in media:

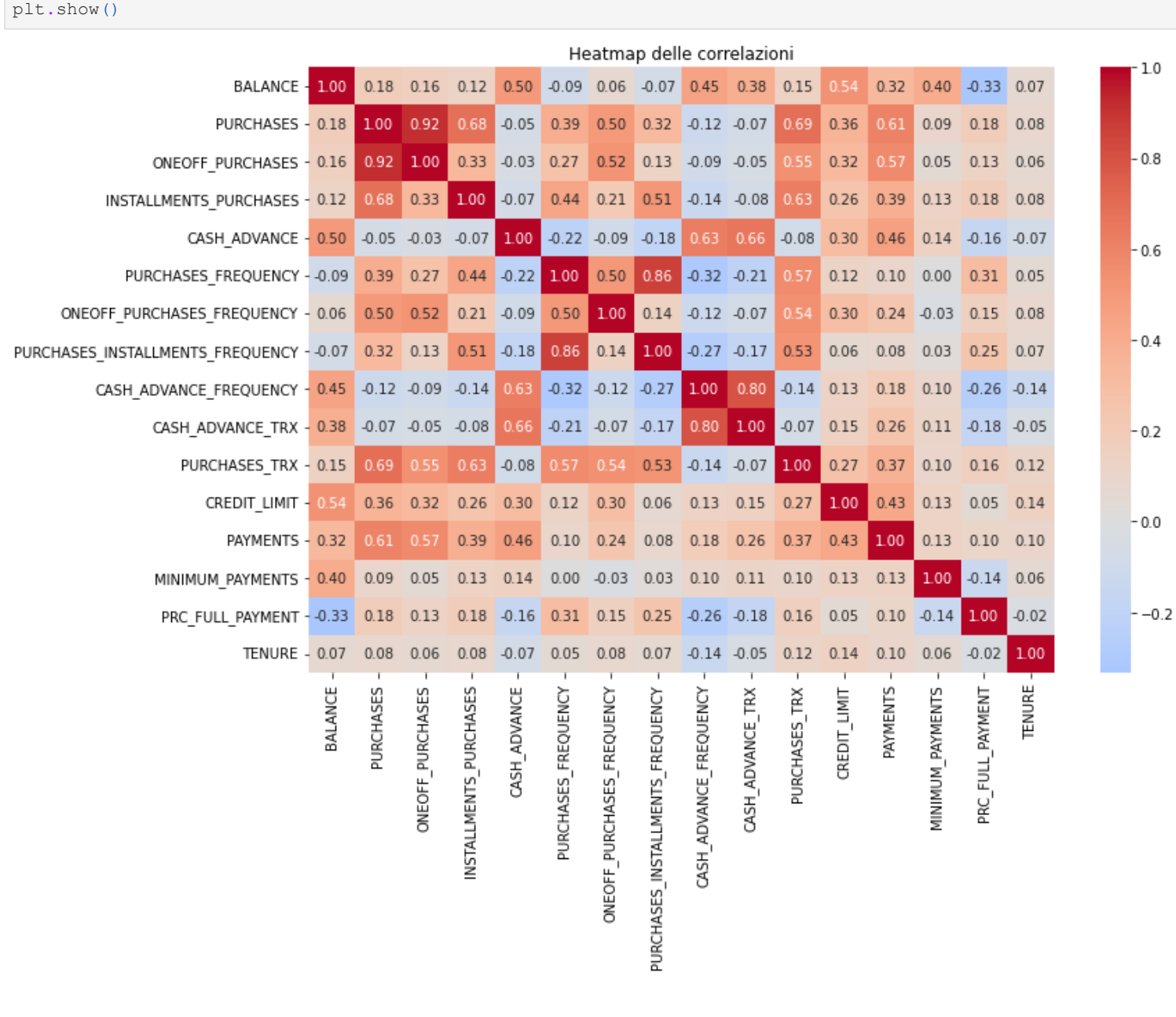
un importo di pagamento totale (PAYMENTS) superiore rispetto a chi aggiorna meno frequentemente, ma anche un minimo pagamento (MINIMUM_PAYMENTS) molto più elevato, e una percentuale di pagamento completo (PRC_FULL_PAYMENT) più bassa.

Questo suggerisce che, pur effettuando pagamenti più alti in termini assoluti, i clienti più attivi potrebbero essere più propensi a pagare solo il minimo richiesto o una parte del saldo, accumulando debito residuo. Al contrario, chi aggiorna raramente il saldo sembra più incline a saldare completamente quanto dovuto, anche se gli importi in gioco sono minori.

CORRELAZIONE TRA VARIABILI NUMERICHE

In [128…

```
# Calcola la matrice di correlazione tra variabili numeriche
# Seleziona solo le colonne numeriche
numeric_cols = data_set_clean.select_dtypes(include='number')

# Calcola la matrice di correlazione solo su queste colonne
corr_matrix = numeric_cols.corr()

# Crea la heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Heatmap delle correlazioni")
plt.show()
```



SELEZIONIAMO LE VARIABILI IN CUI LA CORRELLAZIONE E QUINDI L'INTENSITA DEL LEGAME LINEARE è > 0.7

In [129…

```
# Seleziona solo le colonne numeriche
numeric_cols = data_set_clean.select_dtypes(include='number')

# Calcola la matrice di correlazione solo su queste colonne
corr_matrix = numeric_cols.corr()

# Prendi il valore assoluto delle correlazioni
corr_matrix_abs = corr_matrix.abs()

# Soglia di correlazione
threshold = 0.7

# Escludi la diagonale (correlazione di variabile con se stessa)
mask = ~np.eye(corr_matrix_abs.shape[0], dtype=bool)

# Crea una tabella lunga con solo le correlazioni > threshold
high_corr = (corr_matrix_abs.where(mask)
                          .stack()
                          .reset_index())

high_corr.columns = ['Variabile1', 'Variabile2', 'Correlazione']

# Filtra solo quelle sopra la soglia
high_corr = high_corr[high_corr['Correlazione'] > threshold]

print(high_corr)
```

```
                           Variabile1                        Variabile2  \
16                          PURCHASES                  ONEOFF_PURCHASES   
31                   ONEOFF_PURCHASES                         PURCHASES   
81                PURCHASES_FREQUENCY  PURCHASES_INSTALLMENTS_FREQUENCY   
110  PURCHASES_INSTALLMENTS_FREQUENCY               PURCHASES_FREQUENCY   
128            CASH_ADVANCE_FREQUENCY                  CASH_ADVANCE_TRX   
143                  CASH_ADVANCE_TRX            CASH_ADVANCE_FREQUENCY   

     Correlazione  
16       0.916780  
31       0.916780  
81       0.862338  
110      0.862338  
128      0.799593  
143      0.799593  
```

Come ci si aspetta, il legame lineare tra queste coppie di variabili numeriche è molto elevato.

Cor(CASH_ADVANCE_TRX, CASH_ADVANCE_FREQUENCY) = 0.79

Come ci si aspetta, all' aumentare del numero di transazioni effettuate tramite anticipo di contante, avremo un aumento della relativa frequenza con cui vengono richiesti tali anticipi di contante.

In [130…

```
# Per ridurre la Ridondanza, escludiamo alcune variabile numeriche.

# CUST_ID, BALANCE_FREQUENCY, ONEOFF_PURCHASES, PURCHASES_INSTALLMENTS_FREQUENCY, CASH_ADVANCE_FREQUENCY

# Supponiamo di voler rimuovere queste colonne
cols_to_remove = ['CUST_ID', 'BALANCE_FREQUENCY', 'ONEOFF_PURCHASES', 'PURCHASES_INSTALLMENTS_FREQUENCY', 'CASH_ADVANCE_FREQUENCY']

# Rimuovo colonne
data_set_clean_2 = data_set_clean.drop(columns=cols_to_remove)

scaler = StandardScaler()
scaled_values = scaler.fit_transform(data_set_clean_2)

data_set_scaled = pd.DataFrame(scaled_values, columns=data_set_clean_2.columns)
```

In [131…

```
data_set_scaled.head()
```

Out[131]:

	BALANCE	PURCHASES	INSTALLMENTS_PURCHASES	CASH_ADVANCE	PURCHASES_FREQUENCY	ONEOFF_PURCHASES_FREQUENCY	CASH_ADVANCE_TRX	PURCHASES_TRX	CREDIT_LIMIT	PAYMENTS	MINIMUM_PAYME
0	-0.744625	-0.429184	-0.354826	-0.468655	-0.820769	-0.686280	-0.479437	-0.517623	-0.962575	-0.543942	-0.30
1	0.764152	-0.473208	-0.458839	2.568556	-1.236139	-0.686280	0.099258	-0.597054	0.677204	0.796852	0.08
2	0.426602	-0.116413	-0.458839	-0.468655	1.256077	2.646651	-0.479437	-0.120467	0.813852	-0.399503	-0.09
3	-0.373910	-0.465825	-0.458839	-0.468655	-1.028455	-0.408536	-0.479437	-0.557339	-0.907916	-0.380165	-0.26
4	0.099551	0.142062	0.994815	-0.468655	0.425339	-0.686280	-0.479437	-0.279329	-0.743938	-0.132119	0.65

K-MEANS

K-Means è un algoritmo di clustering che raggruppa i dati in k gruppi (cluster), cercando di rendere i punti all'interno di ogni cluster il più simili possibile tra loro, e il più diversi possibile tra cluster.

Assume cluster circolari e di simile dimensione.

L'inertia misura quanto i punti sono vicini ai centroidi del loro cluster.

Tecnicamente è la somma delle distanze quadratiche (euclidee al quadrato) tra ogni punto e il suo centro. Più bassa è l'inertia, meglio i cluster rappresentano i dati. Aumentando k, l'inertia scende sempre.

In [132…

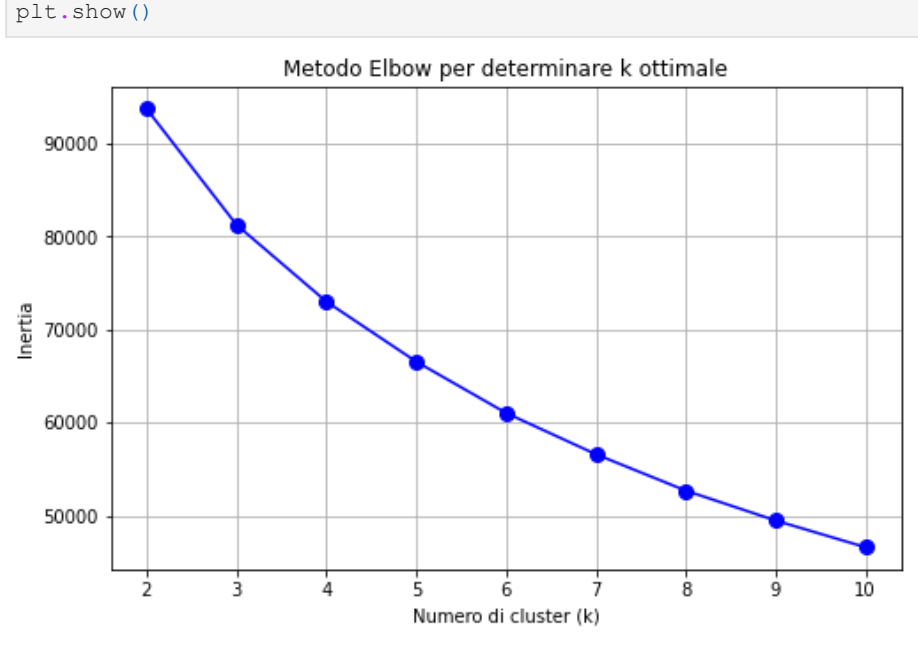
```
# Lista per salvare i valori di inertia
inertias = []
# silhouettes = []
k_range = range(2, 11)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(data_set_scaled)
    # labels = kmeans.labels_
    inertias.append(kmeans.inertia_)
    # score = silhouette_score(data_set_scaled, labels)
    # silhouettes.append(score)
```

Metodo del Gomito (Elbow Method)

Serve per scegliere il numero ottimale di cluster Cerca il "gomito": il punto dove l'inertia smette di calare bruscamente.

In [133…

```
# Plot Elbow
plt.figure(figsize=(8, 5))
plt.plot(k_range, inertias, 'bo-', markersize=8)
plt.xlabel('Numero di cluster (k)')
plt.ylabel('Inertia')
plt.title('Metodo Elbow per determinare k ottimale')
plt.grid(True)
plt.show()
```



Il numero di cluster individuati è pari a 4.

In [134…

```
optimal_k = 4
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(data_set_scaled) + 1
clusters
```

Out[134]:

```
array([2, 4, 1, ..., 2, 2, 1])
```

Aggiungiamo al data_set la colonna clusters

In [135…

```
data_set_clean_2['clusters'] = clusters
```

Raggruppiamo per clusters e facciamo la media di alcune variabili numeriche

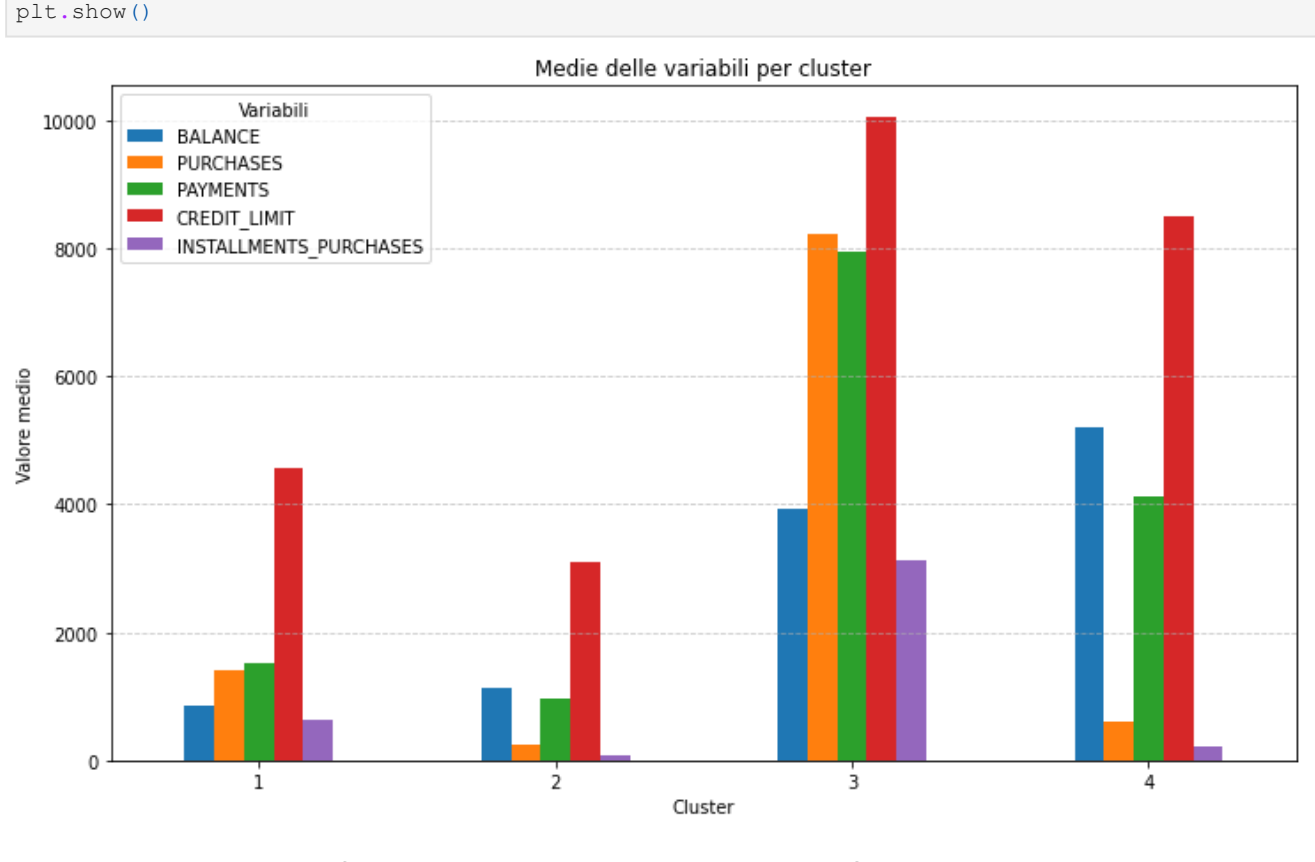
In [136…

```
report = data_set_clean_2.groupby('clusters')[['BALANCE', 'PURCHASES', 'PAYMENTS', 'CREDIT_LIMIT', 'INSTALLMENTS_PURCHASES' ]].mean()
```

Grafico a barre

In [137…

```
# Plot a barre multiple per ogni variabile divisa per cluster
report.plot(kind='bar', figsize=(12, 7))

plt.title('Medie delle variabili per cluster')
plt.xlabel('Cluster')
plt.ylabel('Valore medio')
plt.xticks(rotation=0)  # Mantieni le etichette dei cluster orizzontali
plt.legend(title='Variabili')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Raggruppiamo per clusters e facciamo la media della variabile numerica PRC_FULL_PAYMENT

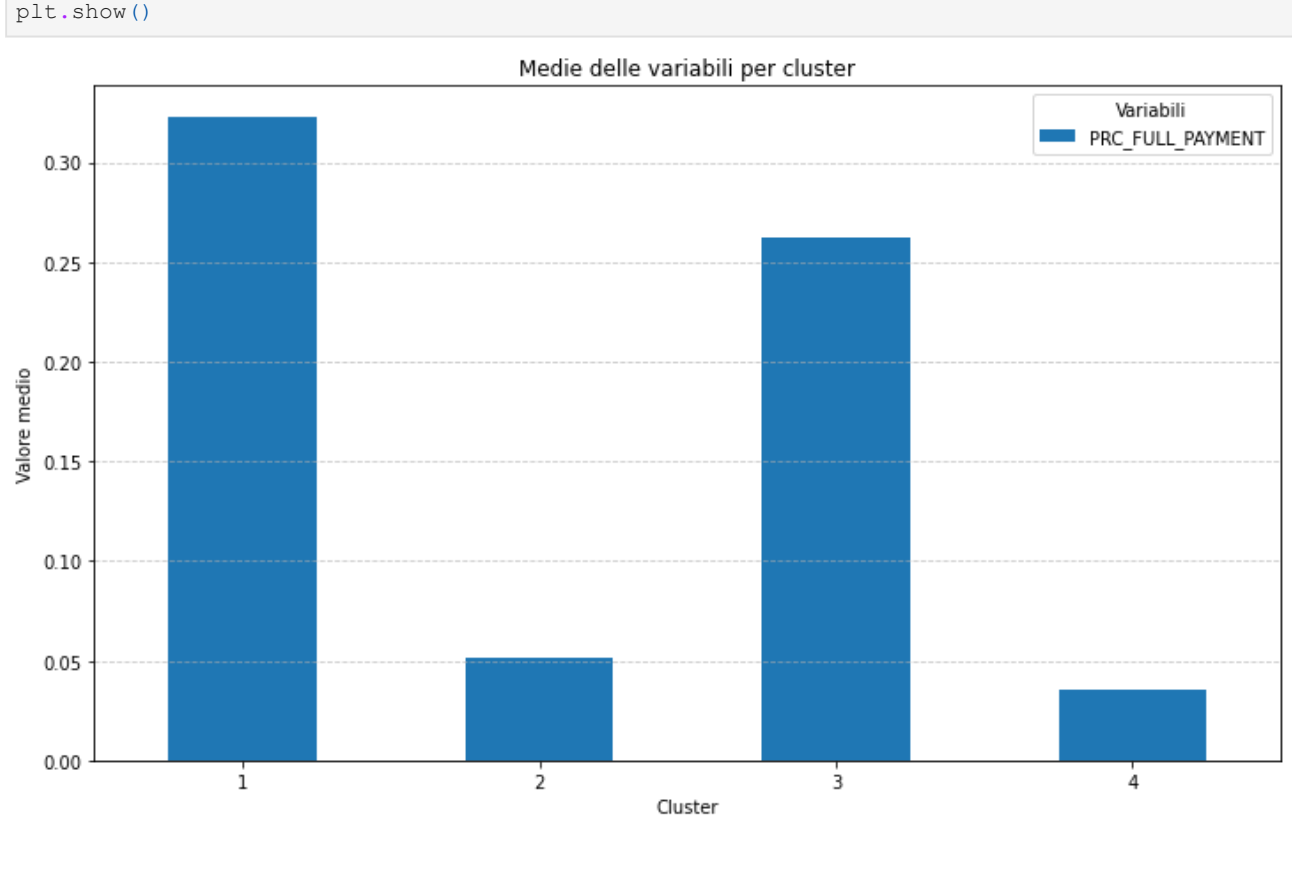
In [138…

```
report_2 = data_set_clean_2.groupby('clusters')[['PRC_FULL_PAYMENT' ]].mean()
```

Grafico a barre

In [139…

```
report_2.plot(kind='bar', figsize=(12, 7))

plt.title('Medie delle variabili per cluster')
plt.xlabel('Cluster')
plt.ylabel('Valore medio')
plt.xticks(rotation=0)  # Mantieni le etichette dei cluster orizzontali
plt.legend(title='Variabili')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



CONSIDERAZIONI

Cluster 1 — Clienti bilanciati e responsabili

Profilo: Clienti con comportamento finanziario sano. Spendono, usano moderatamente le rate, e saldano spesso l'intero debito. Quasi 1 su 3 chiude completamente il debito: clienti a basso rischio, ma anche non particolarmente redditizi dal punto di vista bancario se non si usa il credito rotativo.

Cluster 2 — Clienti passivi con saldo sospetto

Profilo: Clienti poco attivi (poche spese) ma con un saldo residuo rilevante. Pagano parzialmente, ma quasi nessuno salda tutto. Potrebbero rappresentare un profilo rischioso o stagnante, con debiti persistenti e poco coinvolgimento. Da monitorare o attivare con incentivi.

Cluster 3 — Clienti ad alto valore e buon comportamento

Clienti molto attivi. Spendono e rimborsano molto, usano fortemente le rate. Quasi 1 su 4 salda completamente. Cluster premium, interessante sia in termini di profitto che di affidabilità.

Cluster 4 — Saldo elevato, poca attività e molto rischio → clienti potenzialmente sovraindebitati. Pochissimi saldano tutto, usano poco la carta ma mantengono un debito alto. Profilo ad alto rischio. Potrebbero richiedere interventi specifici (es. piani di rientro, blocco credito, ecc.).

K MEDOIDS

K-Medoids è un algoritmo di clustering simile al K-Means, ma più robusto agli outlier.

Si sceglie il numero di cluster k. Invece dei centroidi (media) come in K-Means, ogni cluster è rappresentato da un medoid, ovvero un punto reale del dataset che minimizza la distanza totale dagli altri punti nel cluster.

Ogni punto viene assegnato al medoid più vicino.

I medoid vengono aggiornati fino alla convergenza.

Per un dato numero di cluster k, calcoliamo la Silhouette Score.

Silhouette Score misura quanto bene ogni punto è assegnato al proprio cluster, confrontando:

```
Compattezza: quanto è vicino ai punti del suo stesso cluster

Separazione: quanto è lontano dai punti del cluster più vicino
```

Varia tra -1 e +1:

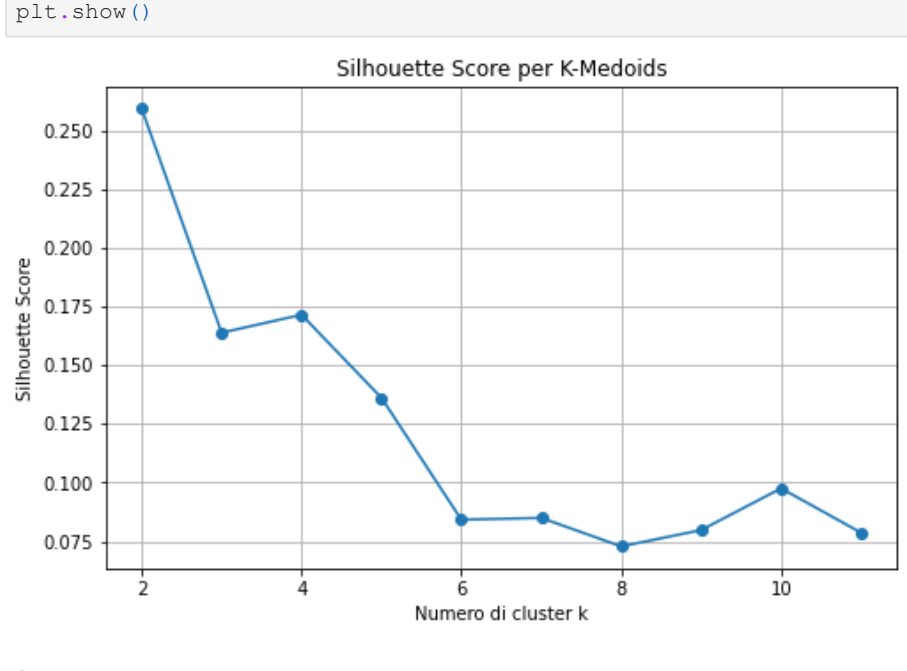
+1 -> Il punto è perfettamente assegnato al suo cluster (ben separato e compatto)

0 -> Il punto è sul confine tra due cluster

-1-> Il punto è stato mal assegnato, più vicino a un altro cluster

In [140…

```
X = data_set_scaled.values

silhouette_scores = []
range_n_clusters = range(2, 12)

for k in range_n_clusters:
    kmedoids = KMedoids(n_clusters=k, random_state=42)
    cluster_labels = kmedoids.fit_predict(X)
    score = silhouette_score(X, cluster_labels)
    silhouette_scores.append(score)
    # print(f'k={k}, Silhouette Score={score:.4f}')

# Plot silhouette scores
plt.figure(figsize=(8,5))
plt.plot(range_n_clusters, silhouette_scores, marker='o')
plt.title('Silhouette Score per K-Medoids')
plt.xlabel('Numero di cluster k')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.show()
```



Selezioniamo il numero ottimale di cluster

In [141…

```
best_k = range_n_clusters[np.argmax(silhouette_scores)]
print(f'Numero ottimale di cluster: {best_k}')
```

```
Numero ottimale di cluster: 2
```

Cluster Agglomerativo

È un metodo gerarchico di clustering bottom-up. All'inizio ogni punto viene considerato come un cluster a se stante, successivamente si uniscono progressivamente in cluster sempre più grandi, fino a uno solo.

Ogni punto è un cluster a sé.

Si calcola la distanza tra tutti i cluster.

Si uniscono i due cluster più vicini (secondo un criterio di "linkage").

Si ripete il processo finché resta un unico cluster.

Tipi di linkage (criteri di unione):

1) Single -> Minima distanza tra due punti nei due cluster

2) Complete -> Massima distanza

3) Average -> Distanza media

4) Ward -> Minimizza la varianza all'interno dei cluster

Risultato: Dendrogramma

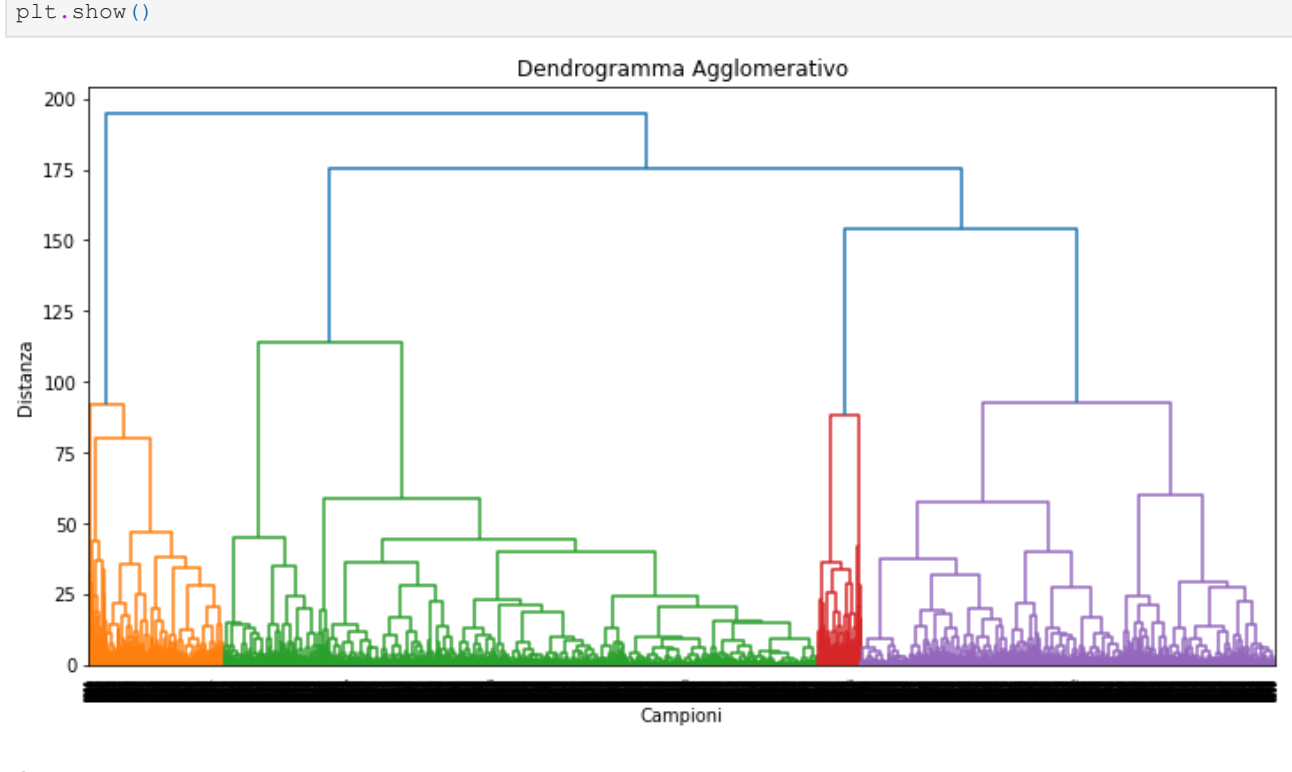
Un dendrogramma mostra visivamente come si formano i cluster man mano che si uniscono.

L'asse y rappresenta la distanza a cui avviene l'unione.

Tagliando orizzontalmente il dendrogramma si scegle il numero di cluster.

In [142…

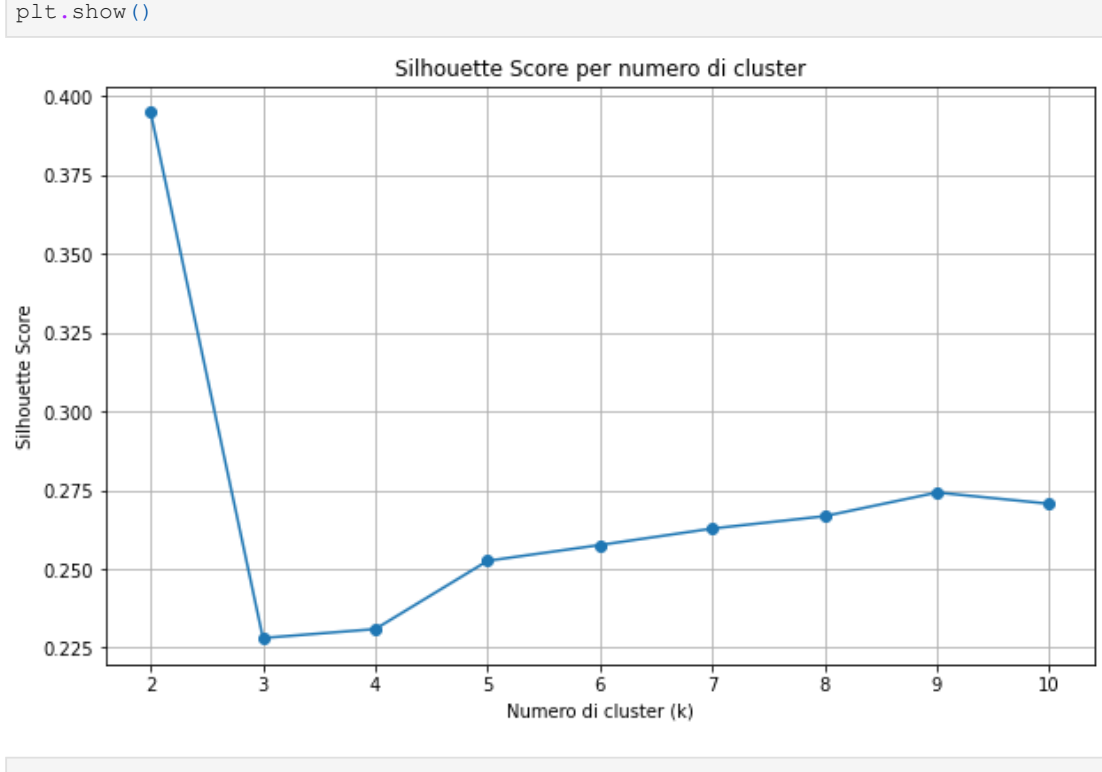
```
# Calcola la linkage (metodo di collegamento)
Z = linkage(data_scaled, method='ward')  # 'ward' minimizza la varianza interna ai cluster

# Plot del dendrogramma
plt.figure(figsize=(12, 6))
dendrogram(Z, truncate_mode='level', p=30)  # p=numero di livelli da mostrare
plt.title('Dendrogramma Agglomerativo')
plt.xlabel('Campioni')
plt.ylabel('Distanza')
plt.show()
```



Considerando come metodo di linkage, il metodo ward, calcoliamo la silhouette score considerando diversi k

In [143…

```
Z = linkage(data_scaled, method='ward')

# Valutazione silhouette per diversi numeri di cluster
silhouette_scores = []
k_values = range(2, 11)  # da 2 a 10 cluster

for k in k_values:
    labels = fcluster(Z, k, criterion='maxclust')  # forza esattamente k cluster
    score = silhouette_score(data_scaled, labels)
    silhouette_scores.append(score)

# Visualizza il grafico silhouette
plt.figure(figsize=(10, 6))
plt.plot(k_values, silhouette_scores, marker='o')
plt.title('Silhouette Score per numero di cluster')
plt.xlabel('Numero di cluster (k)')
plt.ylabel('Silhouette Score')
plt.grid(True)
plt.xticks(k_values)
plt.show()
```



In [144…

```
best_k = k_values[silhouette_scores.index(max(silhouette_scores))]
print(f'Numero ottimale di cluster: {best_k} (Silhouette max = {max(silhouette_scores):.3f})')
```

```
Numero ottimale di cluster: 2 (Silhouette max = 0.395)
```

PCA + K MEANS

La PCA (Principal Component Analysis) è una tecnica di riduzione della dimensionalità che trasforma le variabili originali in un nuovo insieme di variabili non correlate, chiamate componenti principali. Queste componenti sono ordinate in base alla quantità di varianza dei dati che riescono a spiegare.

Impostando n_components = 2, proiettiamo i dati su uno spazio bidimensionale formato dalle prime due componenti principali (PC1 e PC2), che rappresentano le direzioni di massima variabilità e quindi le più informative.

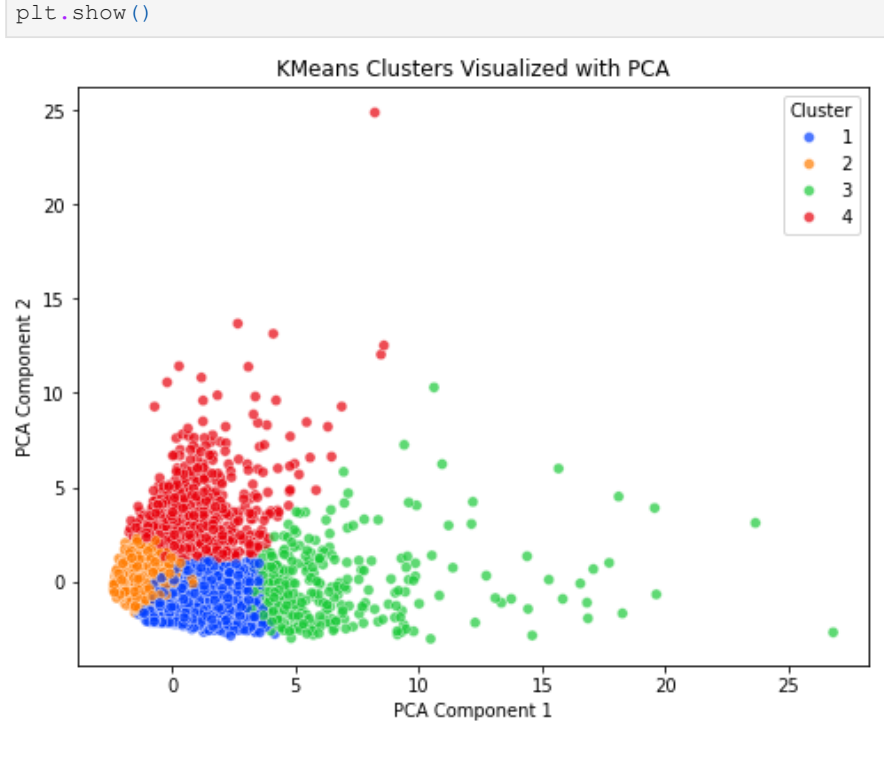
Le componenti principali sono combinazioni lineari delle variabili originali e permettono di sintetizzare l'informazione contenuta nei dati originali in meno dimensioni, mantenendo però la maggior parte della loro struttura.

Successivamente, il clustering (ad esempio con K-Means) viene eseguito su questi dati trasformati, non direttamente sui dati originali.

In questo modo, l'output mostra come i cluster si distribuiscono nello spazio più significativo e informativo possibile, facilitando sia l'identificazione che la visualizzazione dei gruppi.

In [145…

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(data_set_scaled)

plt.figure(figsize=(8, 6))
palette = sns.color_palette("bright", optimal_k)
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=clusters, palette=palette, alpha=0.7)
plt.title('KMeans Clusters Visualized with PCA')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.legend(title='Cluster')
plt.show()
```



Le componenti principali sono combinazioni lineari delle variabili originali, e i loro pesi (o loadings) indicano l'importanza di ciascuna variabile nella costruzione della componente stessa.

Contributo delle variabili a PC1:

PC1 è la direzione di massima varianza nei dati ed è principalmente influenzata da variabili legate agli acquisti e ai pagamenti, come PURCHASES, PURCHASES_TRX (numero di transazioni di acquisto), INSTALLMENTS_PURCHASES e PAYMENTS

Contributo delle variabili a PC2:

PC2 cattura una direzione diversa di variazione, principalmente legata alle variabili relative al CASH_ADVANCE (prelievi di contante) e al BALANCE. Queste variabili hanno i pesi più alti.

In [115…

```
features = data_set_scaled.columns
loadings = pd.DataFrame(
    pca.components_.T,  # trasponi per avere le variabili in righe
    columns=['PC1', 'PC2'],
    index=features
)

# Aggiungiamo una colonna con il valore assoluto per vedere il contributo netto
loadings['Abs_PC1'] = loadings['PC1'].abs()
loadings['Abs_PC2'] = loadings['PC2'].abs()

# Ordiniamo per contributo decrescente
print("Contributo variabili a PC1:")
print(loadings.sort_values(by='Abs_PC1', ascending=False)[['PC1']])

print("\nContributo variabili a PC2:")
print(loadings.sort_values(by='Abs_PC2', ascending=False)[['PC2']])
```

```
Contributo variabili a PC1:
                                 PC1
PURCHASES                   0.445861
PURCHASES_TRX               0.427243
INSTALLMENTS_PURCHASES      0.381989
PAYMENTS                    0.344008
ONEOFF_PURCHASES_FREQUENCY  0.325274
PURCHASES_FREQUENCY         0.303850
CREDIT_LIMIT                0.295251
BALANCE                     0.181681
PRC_FULL_PAYMENT            0.116943
MINIMUM_PAYMENTS            0.101691
TENURE                      0.091127
CASH_ADVANCE                0.058821
CASH_ADVANCE_TRX            0.018019

Contributo variabili a PC2:
                                 PC2
CASH_ADVANCE                0.499720
BALANCE                     0.445557
CASH_ADVANCE_TRX            0.440909
PURCHASES_FREQUENCY        -0.283912
PRC_FULL_PAYMENT           -0.267428
CREDIT_LIMIT                0.236317
PAYMENTS                    0.229655
MINIMUM_PAYMENTS            0.212326
ONEOFF_PURCHASES_FREQUENCY -0.140651
PURCHASES_TRX              -0.130808
INSTALLMENTS_PURCHASES     -0.098156
PURCHASES                  -0.070834
TENURE                     -0.004405
```

La prima componente principale (PC1) spiega il 28.65% della varianza totale dei dati, cioè cattura quasi un terzo dell'informazione complessiva.

La seconda componente principale (PC2) aggiunge un'ulteriore quota di varianza spiegata pari al 20.86%.

Combinando insieme le prime due componenti, otteniamo una rappresentazione che sintetizza circa il 49.52% della varianza totale dei dati.

In altre parole, con solo due nuove variabili (PC1 e PC2) riusciamo a rappresentare quasi la metà dell'informazione presente in tutte le variabili originali, riducendo la dimensionalità senza perdere troppo dettaglio.

In [116…

```
# Percentuale di varianza spiegata da ciascuna componente
explained_variance = pca.explained_variance_ratio_

# Percentuale cumulata per le prime due componenti
varianza_due_componenti = explained_variance[:2].sum() * 100

print(f"Percentuale di varianza spiegata dalla prima componente: {explained_variance[0]*100:.2f}%")
print(f"Percentuale di varianza spiegata dalla seconda componente: {explained_variance[1]*100:.2f}%")
print(f"Percentuale di varianza spiegata dalle prime due componenti: {varianza_due_componenti:.2f}%")
```

```
Percentuale di varianza spiegata dalla prima componente: 28.65%
Percentuale di varianza spiegata dalla seconda componente: 20.86%
Percentuale di varianza spiegata dalle prime due componenti: 49.52%
```